When AES Throughput Depends on the CPU Execution Path: Evidence from x86-64 and Arm64

Marko Mihajlović[1]

Abstract: AES throughput depends on implementation paths and on which software costs a benchmark includes. This paper quantifies how much of an optimized implementation's advantage persists when timing expands from encryption processing to a broader operation. We analyze validated CPU measurements from Windows x86-64 and Oracle Cloud Arm64, supported by separate AES-CBC and AES-GCM campaigns. At 32 MiB, default OpenSSL AES-CTR on Arm64 achieved approximately nineteen times the throughput of the evaluated alternative paths. On Windows, the OpenSSL-to-PyCryptodome throughput ratio decreased from approximately 1.93 at the transform boundary to 1.23 for the instrumented operation. The latter retained 26.7% of OpenSSL's transform throughput and 41.8% of PyCryptodome's. These findings show that cipher-level advantages need not transfer proportionally to surrounding software operations, motivating workload-specific library evaluation with explicit timing boundaries.

Keywords: AES; CPU instruction acceleration; OpenSSL EVP; PyCryptodome; Arm64; authenticated encryption; benchmarking.

1 Introduction

The Advanced Encryption Standard (AES) is a central component of systems that protect stored and transmitted data [1]. Its computational cost matters in applications ranging from bulk storage encryption to services processing short messages. In these settings, encryption shares processor time and memory bandwidth with other application functions. Understanding its performance therefore requires attention both to the cipher implementation and to the software through which an application invokes it.

Modern processors provide dedicated cryptographic instructions, while libraries select among implementations according to processor capabilities and software configuration. However, applications also initialize cipher contexts, allocate output buffers, convert data representations and transfer control between programming languages. A measurement taken within a native encryption routine excludes some of this work. A measurement around a higher-level call includes more of it. Both are useful, but each describes a different part of the computation.

This distinction matters when evaluating native libraries through Python interfaces. Libraries implementing the same AES mode may perform different

[1] Singidunum University, Belgrade, Serbia; E-mail: marko.mihajlovic.23@singimail.rs

preparation and data-handling operations. Their measured difference can consequently reflect cipher implementation, interface design and timer placement. These contributions must be distinguished before using a benchmark to select a library or estimate application capacity.

The central research question is: *how much of an optimized AES implementation's throughput advantage survives when the measurement boundary expands from encryption processing to a broader software operation?* We investigate this question through validated measurements from a Windows x86-64 laptop and an Oracle Cloud Arm64 instance. AES-128 in counter (CTR) mode provides the primary workload. Separate Windows experiments with cipher block chaining (CBC) and Galois/Counter Mode (GCM) provide supporting evidence on key length and authenticated message processing [2, 3].

This study contributes an empirical characterization of the persistence of AES implementation advantages across software measurement boundaries. Two complementary findings support this contribution: the Arm64 results quantify sensitivity to library and capability selection, while the Windows results quantify the attenuation of OpenSSL's relative advantage when broader operation costs are included. CBC and GCM observations establish why this interpretation must also account for message size and cryptographic functionality. The contribution is an evidence-based account of the limits of translating cipher-level performance into software-operation performance, rather than a new AES implementation or a measurement of production application performance.

The analysis uses existing experimental records; no additional measurements were collected for this paper. Comparisons are made within each host and workload, with the timing differences stated explicitly. Section 2 places the study in the context of AES implementation and benchmarking research. Section 3 describes the experimental setup and estimators. Section 4 presents the results and their interpretation, followed by broader implications, limitations and future work in Section 5. Section 6 concludes the paper.

2 Related Work and Research Gap

2.1 AES implementations and workload characteristics

AES uses a fixed block size and supports 128-, 192- and 256-bit keys, with 10, 12 and 14 rounds, respectively [1]. The mode of operation determines how these block transformations are combined. CBC encryption introduces dependencies between successive blocks, whereas CTR generates keystream from independent counter values [2]. GCM combines counter-mode encryption with authentication and includes tag generation or verification [3]. These differences affect the

available parallelism and the work performed per message, so performance must be interpreted for a specified mode and operation.

Research on efficient AES implementations has examined both instruction support and software organization. Käsper and Schwabe [4] presented a bitsliced implementation for 64-bit Intel processors and investigated AES-GCM performance together with resistance to timing attacks. Their work demonstrates that implementation choices matter for both efficiency and security. Intel's AES-NI instructions provide dedicated operations for AES processing [5]. Such mechanisms explain why the same standardized cipher can have substantially different execution costs across implementations, even before application overhead is considered.

Library interfaces add another layer. OpenSSL EVP selects implementations according to library configuration and processor features [6]; PyCryptodome exposes native AES processing through Python [7]. Neither a common algorithm name nor available processor instructions guarantees equivalent code or timing scope. A comparison must therefore identify the actual library configuration and interface.

2.2 Benchmark interpretation and the scope of this study

Systems research has established that apparently minor experimental choices can affect measured performance. Mytkowicz et al. [8] demonstrated measurement bias arising from experimental setup. Kalibera and Jones [9] developed a methodology that accounts for variation at different levels of repetition. These studies motivate documenting the execution environment and distinguishing repeated timings within a session from replication across sessions or machines.

For encryption benchmarks, measurement scope introduces a further interpretive issue. Timing the cipher transformation answers a question about processing prepared data. Timing a surrounding software operation also captures some combination of preparation, allocation, interface overhead and result handling. In authenticated encryption, tag processing and associated data introduce additional work. A throughput comparison is therefore meaningful only when these inclusions are identified alongside the payload size and implementation.

The practical gap addressed here is the persistence of a library advantage when more surrounding software work is timed. We connect implementation research with benchmarking methodology by quantifying both library-path contrasts and their attenuation across measurement boundaries. CBC and GCM provide supporting workload context rather than independent competing contributions.

3 Methodology and Experimental Setup

3.1 Platforms, datasets and correctness checks

The Windows system used an Intel Core i5-10300H (four cores, eight logical processors), Python 3.12.10, PyCryptodome 3.23.0 and verified 64-bit OpenSSL 3.6.5 libraries. The Oracle Cloud instance used a Neoverse-N1-class Arm64 processor with two available virtual CPUs, Python 3.12.3, PyCryptodome 3.23.0 and OpenSSL 3.0.13. Platform metadata, capability settings and source hashes are preserved in the experimental records.

The primary campaigns evaluated AES-128 at 1 KiB, 16 KiB, 1 MiB, 16 MiB and 32 MiB. CTR provides the main workload; archived ECB measurements served as implementation diagnostics. Each configuration used five sessions with seeds 2003–2007, five warm-up repetitions and 20 measured repetitions per session. Known-answer and differential checks established ciphertext correctness before samples were accepted.

Separate earlier Windows campaigns evaluated CBC with 128-, 192- and 256-bit keys and GCM with 128- and 256-bit keys. Both used five seeded sessions, three warm-up repetitions and 20 measured repetitions. We retain the CBC results at 10 MiB and GCM results at 1 KiB, 16 KiB, 1 MiB, 16 MiB and 100 MiB. GCM used 96-bit nonces, 128-bit tags and 16 bytes of AAD, without nonce reuse for distinct messages under one key within a session. Validation included known-answer checks and rejection of modified ciphertext, tags and AAD.

3.2 Implementation paths and timing scopes

Both CTR transform measurements use prepared output buffers. OpenSSL timestamps inside a compiled wrapper delimit EVP Update/Final processing; PyCryptodome's transform interval includes Python dispatch and its internal foreign-function interface. The resulting rates describe different interfaces, rather than isolated AES instruction costs.

The *instrumented operation*, named `operation_envelope` in the records, measures the Python call to the benchmark routine and assignment of its result. It includes input checks, allocation, bindings, setup, transformation, cleanup, output conversion and applicable diagnostics and bookkeeping. The subsequent ciphertext comparison is outside the timer. This boundary is broader than transformation, but is not an uninstrumented application request. Windows provides both intervals for each library.

On Arm64, `OPENSSL_armcap=0` disables multiple optimized capability paths. Default and masked OpenSSL are compared at their common native transform boundary, while the PyCryptodome interval includes additional interface costs.

No instruction trace was collected.

CBC retains its original common timing convention for software and AES-NI paths. GCM includes AAD processing, payload processing, output allocation and tag generation or verification, but excludes construction of a fresh cipher object. Its throughput numerator counts payload bytes only. The three campaigns therefore retain distinct operational definitions.

3.3 Aggregation and interpretation

For a payload of B bytes processed in t nanoseconds, decimal throughput is

$$P = \frac{10^3 B}{t} \quad [\mathrm{MB/s}]. \tag{1}$$

KiB and MiB denote binary payload sizes; MB/s and GB/s denote decimal rates. Each primary result is the median of five session-level rates, calculated as payload bytes divided by the median elapsed time in that session. Session ranges, where reported, describe the smallest and largest of those five medians rather than confidence intervals.

For a reference implementation A and a candidate B measured under a common timing convention, the speedup of B relative to A is

$$S_{B/A}^{(j)} = \frac{\widetilde{t}_A^{(j)}}{\widetilde{t}_B^{(j)}}, \qquad S_{B/A} = \mathrm{median}_{j=1}^{5}\, S_{B/A}^{(j)}. \tag{2}$$

Here, $\widetilde{t}$ denotes the median elapsed time in session j; a value above one indicates faster execution by B. The CBC acceleration factors use this estimator. Approximate CTR throughput ratios discussed below are ratios of the reported aggregate rates, and are distinguished from session-based speedups. Rounding and the order of aggregation can make these estimators differ.

To characterize attenuation, we calculate the OpenSSL/PyCryptodome throughput ratio separately at each boundary. We also calculate, for each library, its instrumented-operation throughput divided by its transform throughput. These retention ratios use unrounded campaign aggregates and describe changes across timing scopes. They are not estimates of the fractions of time spent on allocation, setup or conversion; causal decomposition would require additional component-level measurements.

Sessions, rather than individual repetitions, are the units of replication. Observations are retained regardless of performance direction. Comparisons across hosts, modes and the separate Oracle worker campaigns are descriptive, not statistically paired.

3.4 Archive verification and reproducibility

Reanalysis reads archived data without executing benchmark code. It verifies the selected campaign, five distinct seeds, sample counts, validation status and source hashes. Duplicate single-worker sessions inside the Oracle multicore archive are excluded. Windows contrasts are calculated from unrounded session-summary aggregates.

For Oracle, raw timestamps reconstruct the instrumented-operation boundary in both campaigns. Each worker processes 32 MiB with AES-128-CTR and default OpenSSL. Per repetition, duration is the latest completion minus the earliest start; the numerator is 32 MiB for one worker and 64 MiB for two. Each session rate is this byte count divided by its median duration, followed by the median of five session rates. Native transform rates are neither summed nor compared with these operation rates.

The accompanying `analyze_evidence.py` records archive/member hashes, session identifiers, durations and derived ratios in a machine-readable audit. The `plot_windows_boundaries.py` script checks the Windows input hash and regenerates Fig. 2. These scripts reproduce the numerical contrasts from the preserved records without new measurements.

4 Experimental Results and Interpretation

4.1 Implementation-path sensitivity

Table 1 summarizes AES-128-CTR throughput at 32 MiB. On Windows, the OpenSSL transform rate is 5537 MB/s, compared with 2877 MB/s for PyCryptodome. On Arm64, default OpenSSL reaches 4030 MB/s, whereas the masked OpenSSL configuration and PyCryptodome achieve 218 and 209 MB/s, respectively. These values are medians across five sessions for each configuration.

On Arm64, the default OpenSSL/PyCryptodome throughput ratio is approximately 19.3×, while the default/masked OpenSSL ratio is 18.5×. On Windows, the transform-scope OpenSSL/PyCryptodome contrast is approximately 1.93×. These are implementation/interface throughput contrasts, not isolated AES instruction speedups. In particular, `OPENSSL_armcap=0` changes multiple hardware-optimized execution capabilities rather than selectively disabling AES instructions.

The Arm64 difference is consistent across the five recorded sessions. Default OpenSSL session medians range from 4018 to 4035 MB/s, compared with 217.4–218.4 MB/s for masked OpenSSL and 208.8–209.5 MB/s for PyCryptodome. Thus, the approximately nineteen-fold separation is substantially larger than the observed session variation. It demonstrates sensitivity to implementation and capability

Table 1
AES-128-CTR, 32 MiB: throughput by platform, implementation and timing scope (decimal MB/s).

Host	Implementation	Transform	Full call
Windows i5-10300H	OpenSSL EVP	5537	1479
Windows i5-10300H	PyCryptodome	2877	1202
Oracle Arm64	OpenSSL default	4030	—
Oracle Arm64	OpenSSL armcap=0	218	—
Oracle Arm64	PyCryptodome	209	—

Transform and full-call rates use the intervals defined in Section 3. Full call denotes the instrumented operation. Dashes separate the Arm64 transform comparison from the operation-level worker analysis below.

configuration on this cloud instance, but does not isolate one instruction or establish an architectural advantage of Arm64 over x86-64.

Figure 1 places the 32 MiB results in the context of message size. Larger payloads sustain substantially higher OpenSSL throughput than the alternative paths on each host. At 1 KiB, the native transform interval is very short, making the calculated rate more sensitive to timer resolution and call overhead. Large-buffer throughput should consequently not be used to predict the latency of short requests. The two panels describe separate software environments; differences between them are not estimates of an intrinsic x86-64 versus Arm64 performance ratio.

4.2 Attenuation of implementation advantages

The Windows measurements provide the central comparison across timing boundaries. At 32 MiB, OpenSSL achieves 5537 MB/s for the transform and 1479 MB/s for the instrumented operation; PyCryptodome achieves 2877 and 1202 MB/s, respectively. The OpenSSL/PyCryptodome ratio decreases from approximately 1.93× to 1.23× (unrounded ratios 1.9247 and 1.2299). The broader OpenSSL session medians range from 1452 to 1625 MB/s, while the PyCryptodome medians range from 1179 to 1263 MB/s.

Expressed as a proportion of transform throughput, the broader operation retains 26.7% for OpenSSL and 41.8% for PyCryptodome. Figure 2 distinguishes this within-library retention from the between-library throughput ratio. The former describes how much throughput remains when the timed scope expands; the latter shows how much relative advantage remains over the alternative implementation. OpenSSL remains faster in this benchmark, but its approximately 93% transform-level advantage becomes approximately 23% at the broader boundary.

These observations quantify attenuation without identifying its component

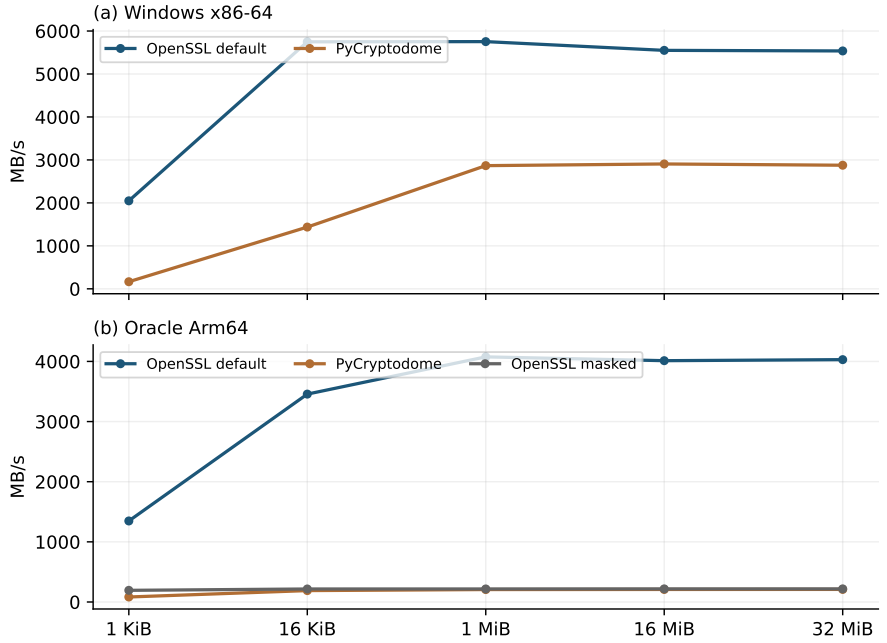
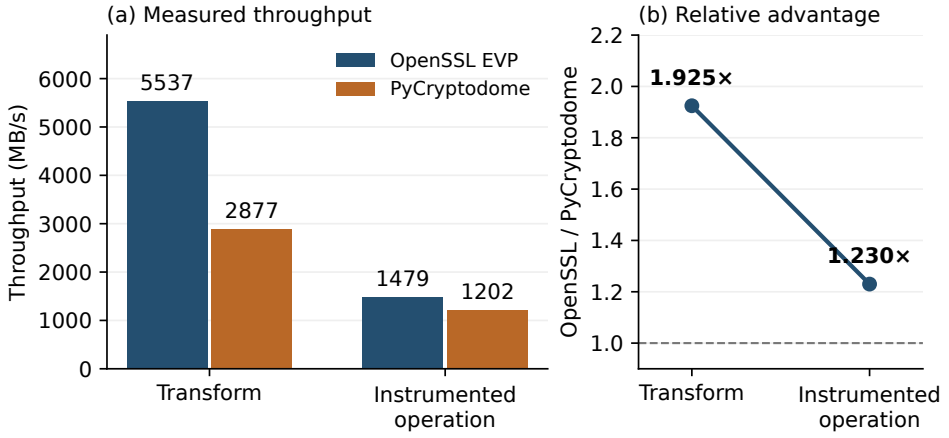


Fig. 1 — AES-128-CTR transform throughput versus payload size on (a) Windows x86-64 and (b) Oracle Arm64. Each panel compares implementations within one host.



Operation / transform throughput: OpenSSL 26.7%; PyCryptodome 41.8%

Fig. 2 — Attenuation of the OpenSSL advantage for Windows AES-128-CTR at 32 MiB. (a) Throughput at each measured boundary. (b) OpenSSL/PyCryptodome ratios calculated from unrounded campaign aggregates. Retention is operation throughput divided by transform throughput, not a component-level overhead measurement.

causes. The native OpenSSL transform excludes the outer Python entry, whereas the PyCryptodome transform includes Python dispatch and its internal foreign-function interface. The broader operation adds preparation, allocation, bindings, setup, result conversion and diagnostics, with implementation-specific costs. The retention ratios cannot establish how much each component contributes, and subtracting aggregate medians would not provide a valid decomposition. In particular, they do not show that allocation alone accounts for the difference or that the library ranking would reverse at another interface.

The practical result is that an implementation advantage measured for prepared encryption processing need not transfer proportionally to a surrounding software operation. The instrumented operation is broader than the transform, but it remains a benchmark call rather than production end-to-end latency. Context reuse, buffer reuse and alternative bindings are consequently candidates for controlled follow-up experiments, not improvements already demonstrated by these measurements.

As a secondary operation-level observation, the verified Oracle archives yield 1059.6 MB/s for one worker and 2031.1 MB/s in aggregate for two workers, a descriptive ratio of approximately 1.92×. Each worker processes 32 MiB, so the two-worker measurement covers 64 MiB across the common completion interval. This is increased aggregate throughput, not a reduction in the latency of one 32 MiB operation. The campaigns comprise separate sessions and are not statistically paired; the ratio is therefore not a controlled causal estimate of multicore scaling efficiency. Both rates use the instrumented-operation boundary defined in Section 3.

4.3 Key length and authenticated processing

As supporting workload evidence, the separate CBC experiment shows how key length affects both absolute throughput and relative acceleration (Table 2). Software encryption decreases from 225.1 MB/s for AES-128 to 174.9 MB/s for AES-256; the AES-NI path decreases from 531.6 to 474.0 MB/s. Nevertheless, the session-based acceleration factor increases from 2.37 to 2.71. Thus, a larger relative benefit from acceleration can coexist with lower absolute throughput. The pattern is consistent with the additional rounds required by longer keys, although the measured factors remain specific to this implementation and mode.

Authenticated processing introduces a different workload. Table 3 reports representative GCM sizes, and Fig. 3 includes all five evaluated sizes. At 1 KiB, AES-128 encryption reaches 70.6 MB/s, while verified decryption reaches 38.1 MB/s. At 1 MiB, the corresponding rates rise to 1148.2 and 1131.4 MB/s. At 100 MiB, the corresponding rates are 1179.7 and 1189.3 MB/s. Encryption generates a tag, whereas authenticated decryption verifies it; their API paths therefore have distinct processing requirements, particularly visible for short payloads.

This pattern is consistent with fixed processing costs being distributed over

Table 2
AES-CBC encryption at 10 MiB on Windows (MB/s). Speedups are medians of session-level time ratios.

Key length	Software	AES-NI path	Speedup
AES-128	225.1	531.6	2.37×
AES-192	198.5	504.3	2.53×
AES-256	174.9	474.0	2.71×

Table 3
AES-GCM payload throughput on Windows (MB/s). Encryption includes tag generation; decryption includes verification. Cipher-object construction is excluded.

Payload	Variant	Encrypt	Verify/decrypt
1 KiB	AES-128-GCM	70.6	38.1
1 KiB	AES-256-GCM	70.1	38.0
1 MiB	AES-128-GCM	1148.2	1131.4
1 MiB	AES-256-GCM	1059.3	1059.8
100 MiB	AES-128-GCM	1179.7	1189.3
100 MiB	AES-256-GCM	1100.4	1080.4

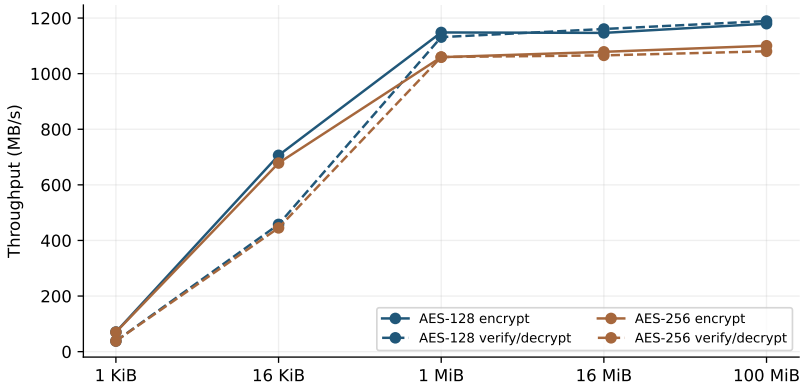


Fig. 3 — AES-GCM encryption and verified-decryption throughput on Windows for five payload sizes. Lines connect the measured aggregate rates.

more payload bytes as message size increases. The plateau at larger sizes suggests that additional payload then increases processing time approximately in proportion to its length over the measured range. AES-256 generally achieves lower throughput than AES-128, as expected from its additional rounds. However, the experiment does not separately time authentication multiplication, verification or allocation, so it cannot assign the observed size dependence to one component.

These GCM results complement the CTR and CBC observations by showing why a benchmark must represent the required cryptographic service. GCM includes authentication, whereas the CTR and CBC workloads reported here measure confidentiality operations. Their throughput figures therefore describe different functionality. These observations originate from separate campaigns with their own timing conventions; they do not measure an incremental GCM-versus-CTR authentication penalty or isolate authentication overhead. The GCM timer also excludes object construction, so a service handling independent messages must additionally account for setup and its surrounding request-processing costs.

5 General Discussion

5.1 Practical implications for CPU encryption systems

The measurements indicate that cryptographic-library selection and operation design should be considered jointly. OpenSSL's substantial transform advantage on Windows becomes smaller when the timer includes the surrounding instrumented operation. The fastest isolated cipher implementation therefore need not deliver a proportionally faster complete request, and a different request design could change the ranking. The observed ranking applies to this benchmark, not a production service.

Selecting an optimized native library matters most when encryption processing occupies a substantial part of the relevant operation. For bulk workloads, the Arm64 contrast makes capability selection an important configuration check. At broader interfaces, allocation, context initialization, conversion and result handling also deserve attention. Engineers should measure their actual interface and payload distribution, then test whether persistent contexts, reusable output buffers or fewer conversions reduce its cost. These remain testable opportunities, not demonstrated improvements.

Short authenticated messages require separate evaluation. The GCM encryption and verified-decryption rates at 1 KiB differ markedly from those at 1 MiB and above, even though cipher-object construction is excluded throughout. Large-buffer throughput consequently cannot characterize complete short-message requests. An application-level evaluation should include the required setup, AAD and tag

operations, as well as the surrounding request processing.

Cloud users should distinguish advertised instruction availability from library configuration and actual execution. A practical audit records the loaded cryptographic library, version, provider and capability overrides, then checks capability settings in that same runtime. OpenSSL exposes CPU settings through its diagnostic interface [10]; a separate command-line installation should not be assumed to represent the library loaded by an application. Correctness checks and interface-specific measurements can reveal slow configurations; instruction tracing is needed to establish the executed path.

5.2 Limitations and scope of generalization

Different processors, operating systems, libraries and runtimes prevent a controlled x86-versus-Arm comparison. Five sessions per configuration are repeated observations on one system, not five independent systems. Scheduling, frequency and cloud contention were not fully controlled, and prepared-buffer reuse differs from cold-cache or streaming workloads. The Oracle worker campaigns are nonpaired and have different aggregate byte counts.

The timing scopes also constrain interpretation. OpenSSL's native transform and PyCryptodome's Python-level transform include different interface costs. The broader operation includes benchmark diagnostics and bookkeeping, so neither throughput retention nor the change in library ratio measures a production request or identifies an individual overhead component. `OPENSSL_armcap=0` changes several capabilities at once, and no instruction-level execution trace was collected. The observed effects of implementation and timing scope are empirical findings; explanations in terms of particular instructions, allocation costs or conversion costs remain hypotheses requiring controlled profiling.

CBC, GCM and CTR originate from separate campaigns with different timing conventions, preventing a matched estimate of authentication cost. Functional validation covers the tested outputs and GCM rejection cases, not side-channel resistance. No complete storage or messaging application, deployed key-management workflow or energy profile was measured.

The strongest generalization concerns experimental design: evaluate the required functionality at the relevant interface and distinguish processing from surrounding-operation costs. Numerical ratios remain specific to the recorded configurations.

5.3 Applications and analogies in other domains

Bulk storage and messaging services should test implementation advantages after preparation and result delivery, using their actual payload distributions.

The Oracle observation also motivates measuring aggregate completion under concurrency separately from single-request latency.

The same measurement issue arises in compression, numerical computing and machine-learning inference. Buffer preparation can surround a fast compression routine, data conversion can precede an optimized numerical kernel, and preprocessing can accompany inference. These are conceptual analogies: a component-level advantage must be evaluated within the larger operation that uses it. No measurements from those domains are claimed here.

For reproducibility, a report should specify the cipher mode, key size, payload distribution, nonce and AAD policy, loaded library and capability configuration, buffer ownership and exact timer placement. It should distinguish per-worker from aggregate throughput, identify the byte count used at each boundary, and preserve session-level observations rather than only the best rate. These details make it possible to reproduce a contrast and assess its relevance to another workload.

5.4 Future research

The first priority is a comparison with matched native entry boundaries for OpenSSL and PyCryptodome, with controlled alternatives for context and output-buffer reuse. This would separate implementation differences from language-interface and preparation costs. Instruction-level profiling on Arm64, combined with selective validation of cryptographic capability settings where supported, would provide stronger evidence about the optimized paths responsible for the observed contrast.

A second priority is equivalent AES-GCM encryption and verified-decryption workloads on Arm64, preserving nonce, AAD and tag policies and measuring setup separately. Complete request-level benchmarks should then use realistic message-size distributions, concurrency and input/output costs to test whether the observed advantages survive within a service. If suitable instrumentation becomes available, energy per processed byte and per completed request would provide an additional criterion alongside throughput and latency. These studies remain future work.

6 Conclusion

This study quantified how much of an AES implementation advantage survives a broader measurement boundary. The Oracle Arm64 results establish strong sensitivity to implementation and capability selection, with default OpenSSL reaching approximately nineteen times the transform throughput of the evaluated alternatives. The central Windows comparison shows that OpenSSL's relative advantage decreases from approximately 1.93× for transformation to 1.23× for

the instrumented operation. Broader-operation throughput retains 26.7% of the OpenSSL transform rate and 41.8% of the PyCryptodome rate, without identifying the costs of individual overhead components.

The contribution is a reproducible characterization of the limits of translating cipher-processing performance into software-operation performance. Supporting CBC and GCM observations reinforce the need to preserve message size and cryptographic functionality when interpreting throughput. Implementation selection should therefore be evaluated together with the operation that invokes it. Matched interfaces, instruction-level profiling and complete request benchmarks are needed to establish causes and application benefits beyond these measured configurations.

References

[1] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023. URL https://doi.org/10.6028/NIST.FIPS.197-upd1.

[2] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001. URL https://doi.org/10.6028/NIST.SP.800-38A.

[3] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007. URL https://doi.org/10.6028/NIST.SP.800-38D.

[4] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant AES-GCM. Cryptology ePrint Archive, Report 2009/129, International Association for Cryptologic Research, 2009. URL https://eprint.iacr.org/2009/129.

[5] Intel Corporation. Intel advanced encryption standard instructions (AES-NI), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[6] OpenSSL Project. EVP symmetric encryption and decryption. OpenSSL documentation, n.d.. URL https://docs.openssl.org/3.0/man3/EVP_EncryptInit/.

[7] PyCryptodome. AES – advanced encryption standard. PyCryptodome documentation, n.d. URL https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html.

[8] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[9] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.

[10] OpenSSL Project. openssl-info: Print OpenSSL built-in information. OpenSSL 3.0 documentation, n.d.. URL https://docs.openssl.org/3.0/man1/openssl-info/.